

Blocco 9 - Gentle Introduction: CTE, viste e manutenibilità

Scomporre query lunghe e dare un nome alle trasformazioni ricorrenti

Relational Databases & SQL

Materiale aggiuntivo

Perché questo materiale

Una introduzione progressiva alla sintassi e al metodo del blocco 9.

Problema di partenza

Situazione concreta

Le query reali diventano presto lunghe: join, filtri, aggregazioni e regole business finiscono nello stesso blocco difficile da leggere.

Ponte con i blocchi precedenti

CTE e viste permettono di dare un nome ai passaggi, come se trasformassimo una spiegazione lunga in paragrafi.

Idea guida

Una CTE è una tabella temporanea logica valida per una singola query.

- ▶ prima si scrive la domanda in linguaggio naturale;
- ▶ poi si decide il soggetto della query;
- ▶ poi si dichiara la granularità del risultato;
- ▶ solo alla fine si rifinisce la sintassi.

Sintassi essenziale

La sintassi è utile solo se rappresenta una decisione chiara sul significato delle righe.

Schema sintattico

```
WITH nome_passaggio AS (  
    SELECT ...  
)  
SELECT ...  
FROM nome_passaggio;
```

Come leggere la sintassi

- ▶ `WITH` introduce una o più CTE.
- ▶ Ogni CTE dovrebbe avere un nome che spiega il passaggio.
- ▶ Una vista salva una query con un nome riusabile.
- ▶ Le CTE aiutano a testare una trasformazione alla volta.
- ▶ La manutenibilità dipende da nomi, granularità e confini chiari.

Dal linguaggio naturale a SQL

Una query è una frase controllabile: soggetto, condizioni, trasformazioni e risultato. Se una parte della frase resta ambigua, anche il codice diventa fragile.

- ▶ il soggetto decide da dove partire;
- ▶ le condizioni decidono cosa includere;
- ▶ le trasformazioni decidono cosa calcolare;
- ▶ l'ordinamento decide come leggere il risultato.

Esempi progressivi

Ogni esempio parte da una richiesta informale, passa per una specifica e arriva alla soluzione SQL.

Esempio 1: Ordini validi come CTE

Domanda

Ordini validi come CTE.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 1: implementazione

```
WITH valid_orders AS (  
  SELECT *  
  FROM order_revenue  
  WHERE status NOT IN ('cancelled', 'refunded')  
)  
SELECT channel,  
       count(*) AS orders,  
       round(sum(gross_revenue), 2) AS revenue  
FROM valid_orders  
GROUP BY channel  
ORDER BY revenue DESC;
```

Esempio 2: Pipeline con più CTE

Domanda

Pipeline con più CTE.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 2: implementazione

```
WITH valid_orders AS (  
    SELECT *  
    FROM order_revenue  
    WHERE status NOT IN ('cancelled', 'refunded')  
),  
orders_with_customer AS (  
    SELECT r.*, c.country, c.segment  
    FROM valid_orders AS r  
    JOIN customers AS c ON c.customer_id = r.customer_id  
)  
SELECT country, segment, count(*) AS orders  
FROM orders_with_customer  
GROUP BY country, segment  
ORDER BY country, segment;
```

Esempio 3: Vista temporanea di sessione

Domanda

Vista temporanea di sessione.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 3: implementazione

```
CREATE TEMP VIEW valid_order_revenue_session AS
SELECT *
FROM order_revenue
WHERE status NOT IN ('cancelled', 'refunded');

SELECT channel, round(sum(gross_revenue), 2) AS revenue
FROM valid_order_revenue_session
GROUP BY channel
ORDER BY revenue DESC;
```

Esercizio guidato

Data una rappresentazione tabellare, costruire la query che risponde alla richiesta.

Rappresentazione tabellare

passaggio	granularità	ruolo
valid_orders	ordine	filtra stati
orders_with_customer	ordine	aggiunge paese
country_kpi	paese	calcola KPI

Problema informale

Richiesta

Data una query lunga su ordini e clienti, riscriverla con CTE leggibili: ordini validi, ordini con cliente, KPI per paese.

Metodo

Evidenziare prima la granularità del risultato, poi scrivere la query.

Errori frequenti

- ▶ usare CTE con nomi generici come `tmp1`;
- ▶ creare una vista senza documentare la granularità;
- ▶ ripetere la stessa logica in molte query;
- ▶ mescolare filtro, join e KPI senza passaggi intermedi;
- ▶ confondere vista logica e tabella fisica.

Checklist finale

- ❶ La domanda informale è stata tradotta in specifica?
- ❷ La granularità del risultato è dichiarata?
- ❸ I filtri e i collegamenti sono motivati?
- ❹ I nomi delle colonne finali sono leggibili?
- ❺ Esiste almeno una query di controllo?

- ▶ scrivere SQL significa rendere verificabile una domanda;
- ▶ ogni costrutto sintattico deve corrispondere a una scelta di modello;
- ▶ esempi piccoli e controlli intermedi evitano errori difficili da vedere.